

```
for i in range(1,n + 1):
    for j in range(1,n + 1):
        plt.plot(i,j, 'o',c = 'black')
plt.savefig(name)
```

In this code, `n` represents the size of each side of our lattice, and we use the `name` argument for the filepath where we want to save the output. The `c = 'black'` argument specifies the color of the points in our lattice. We can create a 5×5 black lattice and save it with the following command:

```
drawlattice(5, 'lattice.png')
```

This is exactly the command that was used to create [Figure 10-1](#).

Representing Games

Since a game of dots and boxes consists of successively drawn line segments, we can record a game as a list of ordered lines. Just as we did in previous chapters, we can represent a line (one move) as a list consisting of two ordered pairs (the ends of the line segment). For example, we can represent the line between (1,2) and (1,1) as this list:

```
[(1,2), (1,1)]
```

A game will be an ordered list of such lines, like the following example:

```
game = [[(1,2), (1,1)], [(3,3), (4,3)], [(1,5), (2,5)], [(1,2), (2,2)], [(2,2), (2,1)],
[(1,1), (2,1)], \ [(3,4), (3,3)], [(3,4), (4,4)]]
```

This game is the one illustrated in [Figure 10-2](#). We can tell it must still be in progress, since not all of the possible line segments have been drawn to fill in the lattice.

We can add to our `drawlattice()` function to create a `drawgame()` function. This function should draw the points of the game board as well as all line segments that have been drawn between them in the game so far. The function in [Listing 10-1](#) will do the trick.

```
def drawgame(n, name, game):
    colors2 = []
    for k in range(0, len(game)):
```